

Atividade Prática: Automação de Pipelines CI/CD com GitHub Actions

Guia de Estudos, Explicação Técnica e Enunciado Prático para Alunos

1. Visão Geral da Atividade

Esta atividade prática tem como objetivo consolidar os conceitos de **Integração Contínua (CI)** e **DevSecOps** por meio da criação e configuração de um pipeline automatizado utilizando o **GitHub Actions**.

Como engenheiros de software ou profissionais de DevOps, é fundamental garantir que o código produzido por uma equipe passe por validações rigorosas de segurança, compilação, estilo e testes funcionais antes de ser integrado ao branch principal de produção. O arquivo de configuração apresentado nesta atividade representa a **solução esperada** que vocês deverão construir e aplicar na aplicação fornecida.

2. Explicação Detalhada do Pipeline Alvo (Solução)

O arquivo de configuração fornecido para a resolução da atividade utiliza o formato YAML e define uma série de automações ativadas por eventos específicos no repositório do Git. Abaixo está a análise de cada seção e configuração contida no pipeline:

Gatilhos Globais (`on`)

O pipeline está configurado para disparar de forma totalmente automática sob duas condições na branch `main`:

- `push` : Toda vez que um desenvolvedor enviar alterações diretamente para a branch `main`.
- `pull_request` : Sempre que uma proposta de junção de código (Pull Request) for aberta ou atualizada tendo a branch `main` como destino.

Estrutura dos Trabalhos (`jobs`)

O pipeline é composto por 4 trabalhos (jobs) organizados estrategicamente. A tabela a seguir descreve a função e as ferramentas mapeadas em cada etapa:

Trabalho (Job)	Tecnologia / Action	Dependência (Needs)	Objetivo Técnico
0. SAST (Segurança)	ajinabraham/ njsscan-action	Nenhuma (Roda em paralelo)	Realizar análise estática de segurança (SAST) no código-fonte em busca de falhas e vulnerabilidades.
1. Build	Node.js v24 & npm run build	Nenhuma (Roda em paralelo)	Baixar dependências da aplicação, configurar o ambiente de execução e validar se o código compila corretamente.
2. Lint & Quality	ESLint & npm run lint	build	Garantir que os padrões de estilo e as boas práticas de escrita de código estejam sendo seguidos rigorosamente.
3. Test	Jest / Ferramenta de testes & npm test	build	Executar a suíte de testes unitários para garantir que nenhuma alteração quebrou as regras de negócio existentes.

Análise dos Blocos de Configuração

- **Isolamento e Paralelismo:** O estágio de `sast` e o de `build` iniciam ao mesmo tempo. Isso otimiza o tempo total de execução da esteira. Se a análise de segurança falhar, o desenvolvedor é notificado imediatamente sem precisar esperar as etapas seguintes.
- **Dependência Lógica (needs):** Os estágios de `lint-and-quality` e `test` possuem a diretiva `needs: build`. Isso significa que eles só iniciarão se o estágio de compilação (Build) terminar com 100% de sucesso. Caso o build falhe, essas etapas são ignoradas, economizando tempo e recursos computacionais.
- **Gerenciamento de Cache:** O parâmetro `cache: 'npm'` na configuração do Node.js faz com que o GitHub armazene as dependências salvas. Em execuções consecutivas, o download dos pacotes se torna muito mais rápido.

3. Exemplo Didático de Referência

Para que você entenda como construir a sua solução sem copiar diretamente a sintaxe do Node.js, analise o exemplo abaixo. Trata-se de um pipeline estruturado para uma **API em Python utilizando FastAPI**. Observe que a lógica estrutural de estágios, dependências e segurança é exatamente a mesma, alterando-se apenas as ferramentas específicas da linguagem:

```

name: Python Product API Pipeline

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  security-sast:
    name: 0. SAST (Segurança Python)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Executar Análise SAST com Bandit
        uses: PyCQA/bandit-action@v1
        with:
          args: "-r src/"

  environment-setup:
    name: 1. Setup & Dependencies
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Configurar o Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Instalar dependências
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

  code-linting:
    name: 2. Code Quality (Flake8)
    needs: environment-setup
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

```

```

- name: Configurar o Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.11'

- name: Instalar Linter
  run: pip install flake8

- name: Executar Linter
  run: flake8 src/

unit-tests:
  name: 3. Automation Tests (PyTest)
  needs: environment-setup
  runs-on: ubuntu-latest
  steps:
    - name: Checkout do código
      uses: actions/checkout@v4

    - name: Configurar o Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'

    - name: Instalar dependências de teste
      run: pip install -r requirements.txt pytest

    - name: Executar Testes Unitários
      run: pytest

```

O que aprender com o exemplo acima? Repare como o job `code-linting` e o job `unit-tests` dependem explicitamente de `environment-setup` através da instrução `needs`. Além disso, o estágio de SAST (usando a ferramenta *Bandit* para Python) roda de forma independente, simulando a mesma arquitetura do pipeline que vocês devem implementar.

4. Proposta da Atividade Prática

Enunciado da Atividade: Implementando a Esteira de CI/CD da Delivery API

Contexto: A equipe de desenvolvimento entregou o código inicial da *Delivery API* (desenvolvida em Node.js), mas o projeto ainda não possui nenhuma automação de integração contínua. Sua missão como especialista em DevOps é criar o arquivo de configuração do GitHub Actions para garantir que nenhum código defeituoso ou inseguro entre em produção.

Instruções Passo a Passo:

1. **Criação do Repositório:** Acesse sua conta no GitHub e crie um novo repositório (público ou privado) com o nome `delivery-api-automation`.
2. **Conexão da Aplicação:** Importe ou clone o código-fonte da aplicação *Delivery API* que foi fornecido pelo professor para a sua máquina local. Configure a sua pasta local para apontar para o novo repositório criado no GitHub utilizando os comandos:

```
git init
git remote add origin <URL_DO_SEU_REPOSITORIO>
git add .
git commit -m "Initial commit: Base Application Code"
```

3. **Estruturação do Workflow:** Na raiz do projeto da aplicação, crie a pasta padrão de automações do GitHub seguindo exatamente o caminho de diretórios: `.github/workflows/`.
4. **Escrita da Solução:** Dentro da pasta `workflows`, crie um arquivo chamado `pipeline.yml`. Escreva e configure o pipeline utilizando as especificações exatas fornecidas na seção de critérios abaixo (que representam a solução robusta para o projeto).
5. **Validação do Pipeline:** Faça o envio do código modificado para a branch `main` do seu GitHub (`git push origin main`). Acesse a aba **Actions** no seu repositório web e acompanhe a execução dos quatro jobs programados.

Critérios de Aceitação e Requisitos do Pipeline:

Para obter a nota máxima, o arquivo `pipeline.yml` deve conter rigorosamente:

- O nome do workflow definido estritamente como `Delivery API Pipeline`.
- Gatilhos configurados para os eventos de `push` e `pull_request` direcionados unicamente à branch `main`.
- O job `sast` configurado com o identificador de segurança `0. SAST (Segurança)`, fazendo uso da biblioteca externa `ajinabraham/njsscan-action@master` informando os argumentos `'.'`.

- O job `build` executando na última versão do Ubuntu estável, configurando o ambiente para o Node.js na versão `'24'` com cache habilitado para o gerenciador `npm`, finalizando com a instalação (`npm install`) e compilação (`npm run build`).
- Os jobs de `lint-and-quality` (rodando `npm run lint`) e `test` (rodando `npm test`) configurados com a declaração de dependência para rodarem apenas após a conclusão com sucesso do job `build`.

Diretrizes de Entrega:

O aluno deverá disponibilizar para avaliação o link público do repositório no GitHub contendo o histórico de commits e a aba Actions com o histórico de execuções bem-sucedidas (todos os jobs sinalizados com o ícone verde de sucesso).